

8-Puzzle Problem using A* Search

Heuristic: number of misplaced tiles. $f(n) = g(n) + h(n)$. Start: 2 8 3 / 1 6 4 / 7 _ 5 Goal: 1 2 3 / 8 _ 4 / 7 6 5

1. Python Code

```
"""8-Puzzle solver using the A* search algorithm.

f(n) = g(n) + h(n)
g(n) : number of moves taken from the start state
h(n) : heuristic = number of misplaced tiles (blank not counted)
"""
import heapq

START = (2, 8, 3,
         1, 6, 4,
         7, 0, 5)

GOAL = (1, 2, 3,
        8, 0, 4,
        7, 6, 5)

MOVES = {"UP": -3, "DOWN": 3, "LEFT": -1, "RIGHT": 1}

def heuristic(state):
    """Misplaced tiles heuristic (blank excluded)."""
    return sum(1 for i in range(9) if state[i] != 0 and state[i] != GOAL[i])

def neighbors(state):
    """Yield (action, new_state, swapped_tile) for every legal move of the blank."""
    b = state.index(0)
    row, col = divmod(b, 3)
    for action, delta in MOVES.items():
        if action == "UP" and row == 0:
            continue
        if action == "DOWN" and row == 2:
            continue
        if action == "LEFT" and col == 0:
            continue
        if action == "RIGHT" and col == 2:
            continue
        n = b + delta
        new = list(state)
        new[b], new[n] = new[n], new[b]
        yield action, tuple(new), state[n]

def astar(start, goal):
    """Return list of (state, g, h, action, tile) from start to goal."""
    counter = 0 # tie-breaker for the heap
    open_list = [(heuristic(start), counter, 0, start)]
    came_from = {start: (None, None, None)}
    best_g = {start: 0}
    closed = set()

    while open_list:
        f, _, g, current = heapq.heappop(open_list)
        if current in closed:
            continue
        if current == goal:
            path = []
            s = current
            while s is not None:
                parent, action, tile = came_from[s]
                path.append((s, best_g[s], heuristic(s), action, tile))
                s = parent
            return path[::-1]
        closed.add(current)

        for action, nxt, tile in neighbors(current):
            ng = g + 1
            if nxt in closed or ng >= best_g.get(nxt, float("inf")):
                continue
            best_g[nxt] = ng
            came_from[nxt] = (current, action, tile)
            counter += 1
```

```

        heapq.heappush(open_list, (ng + heuristic(nxt), counter, ng, nxt))
    return None

def board_str(state):
    sep = " +---+---+---+"
    lines = [sep]
    for r in range(3):
        cells = [" " if v == 0 else f" {v}" for v in state[r * 3:r * 3 + 3]]
        lines.append(" |" + "|".join(cells) + "|")
        lines.append(sep)
    return "\n".join(lines)

def print_step(step, state, g, h, action, tile):
    bar = "=" * 43
    if step == 0:
        title = "INITIAL STATE SETUP"
    else:
        title = f"MOVE BLANK {action} (SWAPPED WITH {tile})"
        if state == GOAL:
            title = f"MOVE BLANK {action} (GOAL STATE REACHED!)"
    print(bar)
    print(f" STEP {step} : {title}")
    print(bar)
    print(board_str(state))
    print()
    print(" COST METRICS:")
    print(f" * Path Cost    g(n) = {g} (moves taken from start)")
    print(f" * Heuristic    h(n) = {h} (estimated remaining distance)")
    print(f" * Total Cost   f(n) = {g + h} (g + h value)")
    print()
    print(" " + "-" * 43)
    print()

if __name__ == "__main__":
    solution = astar(START, GOAL)
    for i, (state, g, h, action, tile) in enumerate(solution):
        print_step(i, state, g, h, action, tile)

    print("#" * 55)
    print(f" FINAL ANSWER: Goal reached successfully at Step {len(solution) - 1}!")
    print(f" Total path cost achieved g(n) = {solution[-1][1]}")
    print("#" * 55)

```

2. Program Output

```
=====
STEP 0 : INITIAL STATE SETUP
=====
+---+---+---+
| 2 | 8 | 3 |
+---+---+---+
| 1 | 6 | 4 |
+---+---+---+
| 7 |   | 5 |
+---+---+---+

COST METRICS:
* Path Cost    g(n) = 0  (moves taken from start)
* Heuristic    h(n) = 4  (estimated remaining distance)
* Total Cost   f(n) = 4  (g + h value)

-----

=====
STEP 1 : MOVE BLANK UP (SWAPPED WITH 6)
=====
+---+---+---+
| 2 | 8 | 3 |
+---+---+---+
| 1 |   | 4 |
+---+---+---+
| 7 | 6 | 5 |
+---+---+---+

COST METRICS:
* Path Cost    g(n) = 1  (moves taken from start)
* Heuristic    h(n) = 3  (estimated remaining distance)
* Total Cost   f(n) = 4  (g + h value)

-----

=====
STEP 2 : MOVE BLANK UP (SWAPPED WITH 8)
=====
+---+---+---+
| 2 |   | 3 |
+---+---+---+
| 1 | 8 | 4 |
+---+---+---+
| 7 | 6 | 5 |
+---+---+---+

COST METRICS:
* Path Cost    g(n) = 2  (moves taken from start)
* Heuristic    h(n) = 3  (estimated remaining distance)
* Total Cost   f(n) = 5  (g + h value)

-----

=====
STEP 3 : MOVE BLANK LEFT (SWAPPED WITH 2)
=====
+---+---+---+
|   | 2 | 3 |
+---+---+---+
| 1 | 8 | 4 |
+---+---+---+
| 7 | 6 | 5 |
+---+---+---+

COST METRICS:
* Path Cost    g(n) = 3  (moves taken from start)
* Heuristic    h(n) = 2  (estimated remaining distance)
* Total Cost   f(n) = 5  (g + h value)

-----

=====
STEP 4 : MOVE BLANK DOWN (SWAPPED WITH 1)
=====
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
```

```
|   | 8 | 4 |
+---+---+---+
| 7 | 6 | 5 |
+---+---+---+
```

COST METRICS:

```
* Path Cost   g(n) = 4 (moves taken from start)
* Heuristic   h(n) = 1 (estimated remaining distance)
* Total Cost  f(n) = 5 (g + h value)
```

```
=====
STEP 5 : MOVE BLANK RIGHT (GOAL STATE REACHED!)
=====
```

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 8 |   | 4 |
+---+---+---+
| 7 | 6 | 5 |
+---+---+---+
```

COST METRICS:

```
* Path Cost   g(n) = 5 (moves taken from start)
* Heuristic   h(n) = 0 (estimated remaining distance)
* Total Cost  f(n) = 5 (g + h value)
```

```
#####
FINAL ANSWER: Goal reached successfully at Step 5!
Total path cost achieved g(n) = 5
#####
```

3. Output Screenshots (Google Colab)

The screenshot shows a Google Colab notebook titled 'Untitled14.ipynb'. The code cell is running, displaying the initial state of a 3x3 puzzle and the cost metrics.

```
print_large_step(s["step"], s["board"], s["g"], s["h"], s["action"])

print("#" * 55)
print(f"FINAL ANSWER: Goal reached successfully at Step 6!")
print(f"Total path cost achieved g(n) = {steps_data[-1]['g']}")
print("#" * 55)
```

STEP 0 : INITIAL STATE SETUP

```
-----
+---+---+
| 2 | 8 | 3 |
+---+---+
| 1 | 6 | 4 |
+---+---+
| 7 | 5 |
+---+---+

COST METRICS:
• Path Cost   g(n) = 0 (moves taken from start)
• Heuristic   h(n) = 6 (estimated remaining distance)
• Total Cost  f(n) = 6 (g + h value)

-----
```

STEP 1 : MOVE BLANK UP (SWAPPED WITH 6)

```
-----
+---+---+
| 2 | 8 | 3 |
+---+---+
| 1 | 1 | 4 |
+---+---+
| 7 | 6 | 5 |
+---+---+

COST METRICS:
• Path Cost   g(n) = 1 (moves taken from start)
• Heuristic   h(n) = 3 (estimated remaining distance)
• Total Cost  f(n) = 4 (g + h value)
```

The Gemini chat interface is visible on the right, showing a greeting: "Hello, Dehini" and a prompt: "How can I help you today?".

The screenshot shows the same Google Colab notebook, but the code cell has progressed to the next step of the puzzle.

```
print_large_step(s["step"], s["board"], s["g"], s["h"], s["action"])

print("#" * 55)
print(f"FINAL ANSWER: Goal reached successfully at Step 6!")
print(f"Total path cost achieved g(n) = {steps_data[-1]['g']}")
print("#" * 55)
```

STEP 3 : MOVE BLANK UP (SWAPPED WITH 2)

```
-----
+---+---+
| 8 | 3 |
+---+---+
| 2 | 1 | 4 |
+---+---+
| 7 | 6 | 5 |
+---+---+

COST METRICS:
• Path Cost   g(n) = 3 (moves taken from start)
• Heuristic   h(n) = 5 (estimated remaining distance)
• Total Cost  f(n) = 8 (g + h value)

-----
```

STEP 4 : MOVE BLANK RIGHT (SWAPPED WITH 8)

```
-----
+---+---+
| 8 | 3 |
+---+---+
| 2 | 1 | 4 |
+---+---+
| 7 | 6 | 5 |
+---+---+

COST METRICS:
• Path Cost   g(n) = 4 (moves taken from start)
```

The Gemini chat interface is visible on the right, showing a greeting: "Hello, Dehini" and a prompt: "How can I help you today?".

colab.research.google.com/drive/1Vw3wVWvRDPkinFpZDIsNTLTPXDOZ5#scrollTo=P0uRzc60uKYk

Continue where you left off: Chrome can restore your tabs every time you restart. To turn this on, go to [Settings](#).

Untitled14.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

...

```
COST METRICS:
• Path Cost    g(n) = 4 (moves taken from start)
• Heuristic    h(n) = 4 (estimated remaining distance)
• Total Cost   f(n) = 8 (g + h value)

=====

STEP 5 : MOVE BLANK DOWN (SWAPPED WITH 2)
=====
+---+---+
| 8 | 2 | 3 |
+---+---+
|   | 4 | 4 |
+---+---+
| 7 | 6 | 5 |
+---+---+

COST METRICS:
• Path Cost    g(n) = 5 (moves taken from start)
• Heuristic    h(n) = 3 (estimated remaining distance)
• Total Cost   f(n) = 8 (g + h value)

=====

STEP 6 : MOVE BLANK RIGHT (GOAL STATE REACHED!)
=====
+---+---+
| 1 | 2 | 3 |
+---+---+
| 8 |   | 4 |
+---+---+
| 7 | 6 | 5 |
+---+---+

COST METRICS:
• Path Cost    g(n) = 6 (moves taken from start)
• Heuristic    h(n) = 0 (estimated remaining distance)
• Total Cost   f(n) = 6 (g + h value)
```

RAM

Disk

Gemini

How can I help you today?

How can I install Python libraries?

Load data from Google Drive

Show an example of training a simple ML model

What can I help you build?

Gemini 3.5 Flash

Variables

Terminal

4:32 PM

Python 3